

P3492R2

Sized deallocation for placement new

Published Proposal, 2025-02-13

Author:

[Lauri Vasama](#)

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Current draft:

vasama.github.io/wg21/D3492.html

Current draft source:

github.com/vasama/wg21/blob/main/D3492.bs

Abstract

Permit the selection of a sized placement deallocation function in new expressions.

Table of Contents

1 Introduction

- 1.1 Usual new and delete expressions
- 1.2 Placement new expressions
- 1.3 Usage with allocators

2 Implementation divergence

3 Proposal

- 3.1 Deduced parameters
- 3.2 Ambiguities

3.3 `std::align_val_t`

3.4 Feature-testing macro

3.5 CWG1628

3.6 Further clarification

4 Alternatives

4.1 Library function

5 Implementation experience

6 Effect on the standard

7 Proposed wording

7.1 [new.syn]

7.2 [expr.new]

7.3 [cpp.predefined]

8 Acknowledgements

References

Informative References

§ 1. Introduction

§ 1.1. Usual new and delete expressions

NOTE: For simplicity these examples ignore any class specific allocation or deallocation functions.

Given the new expression `new T`, the compiler selects an allocation function matching one of the following calls:

- `operator new(sizeof(T))`
- `operator new(sizeof(T), std::align_val_t(alignof(T)))`

Given the delete expression `delete t`, the compiler selects a deallocation function overload matching one of the following calls:

- `operator delete(storage_ptr)`
- `operator delete(storage_ptr, std::align_val_t(alignof(T)))`
- `operator delete(storage_ptr, sizeof(T))`
- `operator delete(storage_ptr, sizeof(T), std::align_val_t(alignof(T)))`

When deallocation functions both with and without size parameters are present, it is unspecified which is selected.

NOTE: In both cases the preference for passing an alignment value depends on whether `T` has new-extended alignment.

Going back to the case of `new T`; if the initialisation of the `T` object were to throw an exception, a *matching* deallocation function is used to deallocate the previously allocated storage. Currently the wording on this is not very clear, but it can be assumed that the intent is to use the deallocation function selected by `delete T`. That might mean selecting a deallocation function not exactly matching the parameters of the used allocation function. For example:

- `operator new(sizeof(T))`
- `operator delete(storage_ptr, sizeof(T))`

Here an additional size parameter is passed to the deallocation function.

§ 1.2. Placement new expressions

Given the placement new expression `new (args...) T`, the compiler selects an allocation function matching one of the following calls:

- `operator new(sizeof(T), args...)`
- `operator new(sizeof(T), std::align_val_t(alignof(T)), args...)`

If the initialisation of the `T` object were to throw an exception now, the storage may be deallocated *if* a matching placement deallocation function is found. In this case a deallocation function matches the allocation function if its parameters, after parameter transformations, are all identical to the parameters of the selected placement allocation function, except for the first parameter which is to be `void*`. It is notable that in the case of failed placement new expressions, no deallocation function containing a size parameter will ever be selected. This is unfortunate for the same reasons explained in [\[N3778\]](#) which introduced the global sized deallocation functions.

Furthermore, unlike global deallocation functions, placement allocation functions are very intentionally provided context for the allocation. Since user allocation schemes are much less constrained than the global allocation and deallocation functions, it is conceivable that a user allocator providing a placement allocation function for ease use might be unable to deallocate the memory without being explicitly provided the size of the allocation. In fact this proposal was created after encountering just that scenario.

NOTE: The terms placement allocation function and placement deallocation function are not currently defined, but we take them to mean that set of allocation functions which are only ever selected by placement new expressions and not by usual new expressions, and the matching set of deallocation functions. See [\[CWG2592\]](#).

§ 1.3. Usage with allocators

The following pattern, using a custom placement allocation function is somewhat common:

```
T* ptr = new (alloc) T(args...);
```

If the construction throws an exception, and a matching placement deallocation function exists, it is invoked to free the allocated memory. That deallocation function does not have access to the size of the allocation. Therefore if `alloc` were to follow the standard allocator model, it would not be possible to deallocate the memory, because the standard allocator model requires providing the size of the allocation to its `deallocate` function.

§ 2. Implementation divergence

There are two cases of implementation divergence:

1. Whether function templates are considered valid candidates and argument deduction is performed. The current standard says no, however EDG alone conforms, and only in the case of global deallocation function templates.

2. The current wording specifies that if a deleted or inaccessible deallocation function is selected, the program is ill-formed. GCC and EDG are non-conforming in this case.

```
template<int>
struct alloc {};

void* operator new(size_t, alloc<0>& a);

template<int I>
void operator delete(void*, alloc<I>& a); // 1

void operator delete(void*, alloc<0>& a) = delete; // 2

int main() {
    alloc<0> al;
    new (al) thrower();
}
```

Current implementation behaviour:

	As shown	With 1 removed	With 2 removed
Clang	Warning: ambiguity	Error: 2 is deleted	Uses 1
GCC	—	—	Uses 1
MSVC	Error: 2 is deleted	Error: 2 is deleted	Uses 1
EDG	—	—	__ ¹

¹ If the operators are class specific, EDG selects an unambiguous function template (i.e. with 1 removed) only if it is not deleted. If the selected function is deleted, no diagnostic is issued.

This proposal provides a drive-by fix and a clarification for the two cases respectively:

1. The current behaviour of GCC, Clang and MSVC becomes conforming. Only EDG is affected.
2. The new example added by this proposal in [\[new.syn\]](#) clarifies this case. This change to the standard is non-normative. GCC, MSVC and EDG are non-conforming in different ways.

§ 3. Proposal

We propose to permit placement new expressions to use two partially matching deallocation functions:

- `operator delete(storage_ptr, std::size_val_t(storage-size), additional-args...)`
- `operator delete(storage_ptr, std::size_val_t(storage-size), std::align_val_t(storage-alignment), additional-args...)`

For the backwards compatibility reasons explained in [P0035R4], which introduced `std::align_val_t`, we propose to use a similar type with the name `std::size_val_t` for passing the size of the allocation. This in conjunction with ignoring overloads with deduced `std::size_val_t` parameters prevents changing the meaning of any existing code.

```
namespace std {
    enum class size_val_t : size_t {};
}
```

Any matching overloads using `std::size_val_t`, if available, are used in preference to those existing ones without `std::size_val_t`. This is because a deallocation function with access to the size is no less efficient than one without, and thanks to the new `std::size_val_t`, no existing code should have its meaning changed. In addition, the use of `std::size_val_t` allows opting into new safer behaviour where ambiguities are no longer silently ignored.

§ 3.1. Deduced parameters

In order to avoid matching existing placement allocation function *templates*, no otherwise matching overload is considered if the type of the `std::size_val_t` parameter was deduced.

Neither of these functions are valid candidates for sized placement deallocation:

```
void operator delete(void* s, auto...);
void operator delete(void* s, std::same_as<std::size_val_t> auto s, my_allocator...);
```

§ 3.2. Ambiguities

In order to avoid accidental memory leaks, if more than one matching placement allocation function with a `std::size_val_t` is found the program is ill-formed.

```
template<typename T>
void operator delete(void* p, std::size_val_t s, my_allocator<T>& a);

void operator delete(void* p, std::size_val_t s, my_allocator<U>& a);

my_allocator<U> alloc;
new (alloc) U; // Ill-formed
```

This is in contrast to existing placement new expressions, where ambiguous placement deallocation functions are simply ignored, as if no matching function was found at all.

§ 3.3. `std::align_val_t`

If the selected placement allocation function contains an implicit `std::align_val_t` parameter, as per the current rules, a matching placement deallocation function must also contain such a parameter. In this case the new `std::size_val_t` parameter is placed directly after the `void*` parameter and before the `std::align_val_t` parameter if any.

```
void* operator new(size_t s, std::align_val_t a, my_allocator& al);

// The only currently matching overload:
void operator delete(void* p, std::align_val_t a, my_allocator& al);

// This proposal allows this overload to be matched:
void operator delete(void* p, std::size_val_t s, std::align_val_t a, my_al

// This overload is never used:
void operator delete(void* p, std::align_val_t a, std::size_val_t s, my_al
```

§ 3.4. Feature-testing macro

Add the following feature test macro: `__cpp_sized_placement_deallocation`

§ 3.5. CWG1628

A partial drive-by fix for the core issue CWG1628 is provided by specifying that the selection of a deallocation function for placement new expressions performs function template argument deduction. This is already the case in GCC, Clang, and MSVC. EDG performs function template argument deduction only for class specific deallocation functions.

§ 3.6. Further clarification

The new wording provided in this proposal further clarifies two cases according to the existing behaviour of implementations:

1. This proposal contains new normative wording specifying that the variadicity of a placement deallocation function must match that of the allocation function in order to be selected. This exactly matches the behaviour of existing implementations:

```
void* operator new(size_t s, alloc_1& al, ...);  
void operator delete(void* p, alloc_1& al); // Never used  
  
void* operator new(size_t s, alloc_2& al);  
void operator delete(void* p, alloc_2& al, ...); // Never used
```

2. This proposal contains a new non-normative note clarifying that additional trailing parameters with default arguments in a deallocation function declaration prevent an exact match. This exactly matches the behaviour of existing implementations:

```
void* operator new(size_t s, alloc& al);  
void operator delete(void* p, alloc& al, int = 0); // Never used
```

§ 4. Alternatives

§ 4.1. Library function

Any discussion of placement new and its deallocation behaviour raises the obvious question of placement deletion. Suppose one uses the placement new syntax to create objects of dynamic storage duration using some custom allocator: `new (alloc) T`. How does one then delete those objects? If

explicit deletion is needed at all, most likely that is achieved using a function template taking the allocator and a pointer to the object to be deleted:

```
template<typename Allocator, typename T>
void delete_via(Allocator const& allocator, T* const ptr)
{
    std::destroy_at(ptr);
    allocator.deallocate(ptr, sizeof(T));
}

delete_via(alloc, new (alloc) T);
```

Or possibly by a more general function template implementing placement delete via calls to placement deallocation functions:

```
delete_via(alloc)( new (alloc) T );
```

Why then, should we not also use a function template for the object creation instead of placement new?

```
template<typename T, typename Allocator, typename... Args>
    requires std::constructible_from<T, Args...>
T* new_via(Allocator const& allocator, Args&&... args);

// new (alloc) T(a, b, c);
new_via<T>(alloc, a, b, c);
```

That is indeed possible and allows one to solve the problem in library, but there are some major drawbacks:

1. Syntax

The placement new syntax provides direct syntactic access to the object initialisation, which has real benefits:

- An `emplace`-like function tends to lose IDE hints pertaining to the constructors of the type `T` being initialised.
- Directly initialising the object allows for more expressive forms of initialisation, such as

designated initialisation:

```
new (alloc) T{ .a = x, .b = y }
```

- The initialisation of `T` requires access to its constructors. If those constructors are not publicly accessible, `new_via` does not have access to them, unless befriended. Direct use of placement new has no such limitation.

These things can still of course be achieved in library using a lambda, but this further degrades the user experience:

```
new_from_result_of(alloc, [&]() { return T{ .a = x, .b = y }; })
```

2. Existing usage

The placement new syntax already exists and is widely used. Instead of inventing a new library alternative, we can improve the performance of code already out there and enable the same code to work correctly with new kinds of user-defined allocators.

§ 5. Implementation experience

This proposal was implemented in Clang at github.com/vasama/llvm (branch P3492).

This implementation was tested against large real world code bases that make extensive use of placement allocation functions and function templates, including function templates containing deduced parameter packs.

§ 6. Effect on the standard

- Adds one scoped enum in the standard library: `std::size_val_t`.
- Changes the selection of a deallocation function only for placement new expressions.
- Adds a feature-testing macro: `__cpp_sized_placement_deallocation`.
- Partially solves [CWG1628] by applying template argument deduction to placement deallocation function templates.

§ 7. Proposed wording

§ 7.1. [new.syn]

Modify [new.syn] as follows:

```
namespace std {  
    // placement operator new control  
    enum class size_val_t : size_t {};  
}
```

§ 7.2. [expr.new]

Modify [expr.new] as follows:

If the *new-expression* does not begin with a unary `::` operator and the allocated type is a class type `T` or an array thereof, a search is performed for the deallocation function's name in the scope of `T`. Otherwise, or if nothing is found, the deallocation function's name is looked up by searching for it in the global scope. The set of lookup results are the *candidates* used in the placement deallocation function selection process.

For a placement allocation function, the selection of a placement deallocation function is first attempted using a prvalue argument of type `std::size_val_t` immediately following the first argument. This is called the *size argument*. If the selection process terminates without having selected a deallocation function, the process is restarted without the size argument. The first argument together with the size argument (if any) are called the *implicit arguments*.

A placement deallocation function is selected as follows:

- If a size argument is used, eliminate from further consideration any templated functions where the type of the second parameter is dependent.
- In each case where a candidate is a function template, candidate function template specializations are generated using template argument deduction ([temp.over], [temp.deduct]). The arguments used for the deduction are the implicit arguments followed by the additional arguments.
- Eliminate from further consideration any functions whose *parameter-declaration-clause* terminates with an ellipsis, but that of the allocation function does not.
- Eliminate from further consideration any functions whose *parameter-declaration-clause* does not terminate with an ellipsis, but that of the allocation function does.
- Eliminate from further consideration any functions where the number of parameters is not equal to the sum of the number of implicit arguments and the number of additional arguments.
- Eliminate from further consideration any functions where, after parameter transformations ([dcl.fct]), the types of the function parameters are not identical to the types of the implicit arguments followed by the types of the parameters of the allocation function except its first parameter.
- If exactly one function remains, that function is selected and the selection process terminates.
- Otherwise, if an implicit size argument was used, the program is ill-formed.
- Otherwise, the selection process terminates without having selected a deallocation function.

[Note: A deallocation function with an additional trailing parameter compared to the allocation function is never matched, even if a default argument is provided. —end note]

A declaration of a placement deallocation function matches the declaration of a placement allocation function if it has the same number of parameters and, after parameter transformations ([`decl.fct`]), all parameter types except the first are identical. If the lookup finds a single matching deallocation function, that function will be called; otherwise, no deallocation function will be called. If the lookup finds a usual deallocation function and that function, considered as a placement deallocation function, would have been selected as a match for the allocation function, the program is ill-formed. For a non-placement allocation function, the normal deallocation function lookup is used to find the matching deallocation function ([`expr.delete`]). In any case, the matching deallocation function (if any) shall be non-deleted and accessible from the point where the *new-expression* appears.

The following examples are entirely new, but are not highlighted in green in order to improve readability:

[Example:

```
struct A {};  
struct T {};  
  
void* operator new(std::size_t s, A& al); // #1  
void operator delete(void* p, A& al); // #2  
void operator delete(void* p, std::size_val_t x, A& al); // #3  
  
A al;  
new (al) T(); // Uses #1 and #3.
```

—end example]

[Example:

```
template<int I>  
struct A {};  
struct T {};  
  
void* operator new(std::size_t s, A& al);  
  
template<int I>  
void operator delete(void* p, std::size_val_t x, A<I>& al);  
void operator delete(void* p, std::size_val_t x, A<0>& al);  
  
A<0> al;  
new (al) T(); // error: ambiguous deallocation function
```

—end example]

[Example:

```
struct A {};  
struct T {};  
  
void* operator new(std::size_t s, A& al);  
void operator delete(void* p, A& al) = delete;  
  
A al;  
new (al) T(); // error: attempted to use deleted function
```

—end example]

If a *new-expression* calls a deallocation function, it passes the value returned from the allocation function call as the first argument of type `void*`. If a placement deallocation function with a parameter corresponding to an implicit size argument is called, it is passed the size argument used for the allocation function converted to `std::size_val_t`. If a placement deallocation function is called, it is passed the same additional arguments as were passed to the placement allocation function, that is, the same arguments as those specified with the *new-placement syntax*. If the implementation is allowed to introduce a temporary object or make a copy of any argument as part of the call to the allocation function, it is unspecified whether the same object is used in the call to both the allocation and deallocation functions.

§ 7.3. [cpp.predefined]

Add the following row to the table in [cpp.predefined]:

<code>__cpp_sized_placement_deallocation</code>	202502L
---	---------

§ 8. Acknowledgements

Big thanks to Oliver Hunt for the help.

§ References

§ Informative References

[CWG1628]

Richard Smith. *Deallocation function templates*. 22 February 2013. open. URL: <https://wg21.link/cwg1628>

[CWG2592]

Jim X. *Missing definition for placement allocation/deallocation function*. 14 April 2022. open. URL: <https://wg21.link/cwg2592>

[N3778]

Lawrence Crowl. *C++ Sized Deallocation*. 27 September 2013. URL: <https://wg21.link/n3778>

[P0035R4]

Clark Nelson. *Dynamic memory allocation for over-aligned data*. 21 June 2016. URL: <https://wg21.link/p0035r4>